Optimización de la Red Logística de Recepción y Procesamiento de Residuos Agropecuarios con el Método de Transporte y un Algoritmo Genético

Carlos Mario González Rico — 160005042
Juan Francisco Lemus — 160005014
Jaider Adrián Traslaviña — 160005044

Programa de Ingeniería de Sistemas — Curso de Optimización — Universidad de los Llanos

***Resumen*—** EcoResiduos del Llano S.A.S. recoge los residuos orgánicos de diez empresas agropecuarias del Meta y los procesa en cuatro plantas ubicadas en Villavicencio, Restrepo, Granada y Puerto Gaitán. Este informe modela esa red como un problema de transporte de costo mínimo y lo resuelve con dos enfoques programados por el equipo en Python, sin librerías de optimización: un método exacto que parte de la aproximación de Vogel y llega al óptimo con el método MODI, y un algoritmo genético con codificación real, decodificación proporcional y aptitud penalizada por exceso de capacidad. Con los datos de prueba regionales el costo óptimo es $33.263.400 por semana, que MODI alcanza en 2 iteraciones en menos de un milisegundo. El algoritmo genético con su configuración base queda a 0,003 % del óptimo en su mejor corrida y a 0,52 % en promedio sobre 30 corridas. El análisis de sensibilidad muestra que las plantas de Restrepo y Granada son el cuello de botella: cada tonelada adicional de capacidad ahorraría $18.000 y $16.000 por semana, respectivamente. El sistema se construyó en tres capas: interfaz en Streamlit, lógica en FastAPI y datos en SQLite.

***Palabras clave*—** Problema de transporte, método de Vogel, método MODI, algoritmos genéticos, análisis de sensibilidad, logística de residuos agropecuarios.

I. INTRODUCCIÓN

A. Contexto y problema

EcoResiduos del Llano S.A.S. transforma residuos agropecuarios (biomasa y subproductos de palma, arroz, ganadería y cultivos) en abonos orgánicos, biocombustibles y suplementos para nutrición animal [1]. Su red integra diez empresas agropecuarias aliadas, que generan un volumen semanal constante de residuos, y cuatro plantas de recepción y transformación.

Hoy la asignación de rutas se decide de forma empírica, lo que produce sobrecostos de transporte. La gerencia necesita saber cuántas toneladas debe enviar cada empresa a cada planta para minimizar el costo total, poder agregar empresas o plantas y modificar los coeficientes del negocio, y entender cómo responde la solución ante cambios en la oferta, la capacidad y los costos.

El problema se formula como un modelo de transporte y se resuelve de dos formas: con un método exacto que garantiza el óptimo global y con un algoritmo genético cuyo desempeño se mide contra ese óptimo. La sección II presenta el equipo; la III, la formulación; la IV y la V, los métodos de solución; la VI, la arquitectura del software; la VII, los datos de prueba; la VIII y la IX, los resultados y el análisis de sensibilidad; la X, el análisis comparativo, y la XI, las conclusiones.

B. Objetivos

El objetivo general es diseñar y desarrollar una solución computacional basada en modelos de redes, métodos de transporte y algoritmos genéticos que minimice el costo logístico de la red de recepción de residuos. Los objetivos específicos son:

- Estructurar y parametrizar el problema con la oferta de las diez empresas y la capacidad de las cuatro plantas.
- Implementar un método exacto que encuentre la solución óptima global.
- Diseñar e implementar un algoritmo genético adaptado a la asignación de flujos de la red.
- Analizar la sensibilidad de la solución ante variaciones en costos, oferta y capacidad.
- Comparar ambos métodos en calidad de la solución, tiempo de cómputo, escalabilidad y respuesta ante escenarios.

II. DEFINICIÓN DEL GRUPO Y ASIGNACIÓN DE ROLES

El proyecto lo desarrolla un grupo de tres integrantes. Cada uno asume un rol durante todo el ciclo del proyecto, con responsabilidades sobre una parte del sistema y del informe, como resume la Tabla I. Las decisiones que afectan a más de un módulo, como la interfaz común ISolver o el formato de los datos que viajan entre la API y la interfaz, se acuerdan entre los tres.

TABLA I
ROLES DEL EQUIPO DE TRABAJO

Integrante	Rol	Responsabilidades
Jaider Adrián Traslaviña (160005044)	Líder técnico y desarrollador de metaheurísticas	Coordina el trabajo técnico y las integraciones. Diseña, programa y ajusta el algoritmo genético y corre los experimentos de comparación.
Carlos Mario González Rico (160005042)	Ingeniero de modelado y optimización	Formula el modelo matemático, programa el método exacto (Vogel y MODI), el análisis de sensibilidad y los datos de prueba regionales.
Juan Francisco Lemus (160005014)	Desarrollador de software e interfaz	Define la arquitectura en tres capas, la API, la persistencia y la interfaz en Streamlit, y elabora los diagramas de aplicaciones, clases y datos.

III. FORMULACIÓN MATEMÁTICA

A. La red como grafo

La red se representa como un grafo dirigido $G = (V, E)$, con $V = F \cup P$. El conjunto $F = \{1, \ldots, n\}$ contiene las empresas agropecuarias (fuentes) y $P = \{1, \ldots, m\}$, las plantas (destinos); en el caso base, $n = 10$ y $m = 4$. Cada arco $(i, j) \in E$ es la ruta de la empresa i a la planta j. Como todos los arcos van de una fuente a un destino, el grafo es bipartito y el problema corresponde al modelo de transporte de costo mínimo [2], [3].

B. Parámetros

- a_i: residuos que genera la empresa i, en toneladas por semana (oferta).
- b_j: capacidad de procesamiento de la planta j, en toneladas por semana.
- d_{ij}: distancia de la ruta (i, j), en kilómetros.
- t: tarifa de transporte, en pesos por tonelada-kilómetro.
- o_j: costo de operación de la planta j, en pesos por tonelada.

El costo unitario de cada ruta suma el transporte y la operación de la planta:

$$c_{ij} = d_{ij} \cdot t + o_j \tag{1}$$

Si una ruta no está disponible, su costo se fija en un valor M muy grande (método de la Gran M, $M = 10^8$), y si una planta está cerrada su capacidad es $b_j = 0$.

C. Variables de decisión

La variable $x_{ij} \geq 0$ representa las toneladas semanales que la empresa i envía a la planta j. El caso base tiene $n \cdot m = 40$ variables.

D. Función objetivo

Se minimiza el costo total semanal de transporte y operación:

$$\min Z = \Sigma_{i \in F} \Sigma_{j \in P} c_{ij} x_{ij} \tag{2}$$

E. Restricciones

Cada empresa entrega todos sus residuos, ninguna planta supera su capacidad y los flujos no son negativos:

$$\Sigma_{j \in P} x_{ij} = a_i, \quad \forall\, i \in F \tag{3}$$

$$\Sigma_{i \in F} x_{ij} \leq b_j, \quad \forall\, j \in P \tag{4}$$

$$x_{ij} \geq 0, \quad \forall\, i \in F, j \in P \tag{5}$$

F. Factibilidad y balanceo

El modelo tiene solución si y solo si $\Sigma\, a_i \leq \Sigma\, b_j$, es decir, si la capacidad total alcanza para toda la oferta; el sistema valida esta condición antes de resolver. En el caso base la oferta es de 373 t y la capacidad de 420 t por semana. Para aplicar los métodos de tabla, que trabajan con igualdades, se agrega una fuente ficticia con oferta $\Sigma\, b_j - \Sigma\, a_i = 47$ t y costo cero. Lo que esa fila envía a una planta es capacidad que queda sin usar.

IV. MÉTODO EXACTO: VOGEL Y MODI

El método exacto se programó sin librerías de optimización, siguiendo el procedimiento clásico del problema de transporte [2], [3]. Sus etapas aparecen en el diagrama de secuencia de la Fig. 11 (Anexo A).

A. Solución inicial por el método de Vogel

Para cada fila y cada columna activa se calcula una penalización igual a la diferencia entre sus dos costos más bajos. Se elige la fila o la columna con la mayor penalización, se asigna a su celda más barata la mayor cantidad posible, min(oferta, demanda), y se tacha la fila o la columna satisfecha. El proceso se repite hasta cubrir toda la oferta [4]. Vogel suele partir más cerca del óptimo que la esquina noroeste o el costo mínimo, por lo que MODI necesita menos iteraciones.

B. Optimización con el método MODI

Con la solución básica actual se calculan los potenciales u_i y v_j resolviendo $u_i + v_j = c_{ij}$ en las celdas básicas, con el potencial de la fila ficticia fijado en cero. Después se obtiene el costo reducido de cada celda no básica:

$$\bar{d}_{ij} = c_{ij} - u_i - v_j \tag{6}$$

Si todos los $\bar{d}_{ij}$ son mayores o iguales a cero, la solución es óptima. Si no, entra a la base la celda con el $\bar{d}_{ij}$ más negativo, se forma el ciclo cerrado que la une con celdas básicas alternando signos + y −, y se desplaza por el ciclo una cantidad θ igual a la menor asignación de las celdas con signo −; la celda que queda en cero sale de la base [5]. En el código la base se maneja como un árbol de expansión sobre filas y columnas (clase ArbolBase), que entrega tanto los potenciales como el ciclo de cada celda entrante.

C. Degeneración

Una tabla balanceada con r filas y s columnas debe tener $r + s - 1$ celdas básicas. Si Vogel satisface una fila y una columna al mismo tiempo quedan menos; en ese caso se agregan celdas básicas con asignación cero que no formen ciclos (se verifica con una estructura de conjuntos disjuntos), para que los potenciales queden definidos.

D. Verificación

Antes de entregar la respuesta se elimina la fuente ficticia y se comprueba que la solución cumple (3) a (5). El resultado incluye la matriz de flujos, el costo total, el tiempo de cómputo, el número de iteraciones y los valores duales que usa el análisis de sensibilidad. La implementación se validó contra un solucionador de programación lineal en 400 instancias aleatorias, con el mismo costo óptimo en todas.

V. ALGORITMO GENÉTICO: DISEÑO Y JUSTIFICACIÓN

El algoritmo genético (AG) es una metaheurística inspirada en la selección natural [6], [7]: mantiene una población de soluciones y la mejora generación tras generación mediante selección, cruce y mutación. No garantiza el óptimo, por lo que su calidad se mide contra la solución exacta. La Fig. 12 (Anexo A) muestra su secuencia.

A. Codificación del cromosoma

Cada individuo es un vector de $n \cdot m$ genes reales en [0, 1], organizado en n bloques de m genes, uno por empresa. El gen

g_{ij} expresa la preferencia relativa de la empresa i por la planta j.

B. Decodificación

Los flujos se obtienen repartiendo la oferta de cada empresa en proporción a sus genes, sin contar las rutas bloqueadas:

$$x_{ij} = a_i \cdot g_{ij} / \Sigma_{k \in P} g_{ik} \quad (7)$$

Con esta decodificación toda solución cumple (3) y (5) por construcción, y el algoritmo solo debe controlar la restricción de capacidad (4). La idea sigue el tratamiento evolutivo del problema de transporte de Michalewicz [8].

C. Función de aptitud

La aptitud es el inverso del costo total penalizado por exceder la capacidad de las plantas:

$$f = 1 / (1 + Z(x) + \lambda \cdot \Sigma_{j \in P} \max(0, \Sigma_i x_{ij} - b_j)) \quad (8)$$

Por defecto λ es el doble del mayor costo unitario de las rutas habilitadas. Así, enviar una tonelada por encima de la capacidad de una planta siempre cuesta más que redirigirla a cualquier otra.

D. Operadores genéticos

- **Selección:** por torneo de k individuos, en el que gana el de mayor aptitud, o por ruleta, con probabilidad proporcional a la aptitud.
- **Cruce:** de uno o de dos puntos sobre el vector de genes, con probabilidad p_c.
- **Mutación:** uniforme, que reemplaza un gen por un valor aleatorio, o de poda, que con probabilidad 1/2 lo lleva a cero (cierra esa ruta) y si no lo reemplaza. Se aplica a cada gen con probabilidad p_m.
- **Elitismo:** los mejores individuos pasan sin cambios a la generación siguiente.

E. Parámetros

Los parámetros se ajustan desde la interfaz y se envían a la API en cada comparación. La Tabla II muestra los valores por defecto de la API, los de la interfaz y los rangos permitidos. Cada corrida usa una semilla distinta, de modo que los resultados son reproducibles.

TABLA II
PARÁMETROS DEL ALGORITMO GENÉTICO

Parámetro	API	Interfaz	Rango en la interfaz
Tamaño de la población	100	80	20–300
Generaciones	300	150	20–600
Probabilidad de cruce p_c	0,80	0,85	0–1
Probabilidad de mutación p_m	0,05	0,05	0–0,30
Individuos élite	2	2	0–20
Tamaño del torneo k	3	3	2–10
Selección / cruce / mutación	torneo / 1 punto / poda	igual	torneo o ruleta; 1 o 2 puntos; uniforme o poda
Corridas por comparación	10	10	1–30 (API: 1–50)

F. Justificación del diseño

La codificación real con decodificación proporcional evita generar individuos que incumplan la oferta. Con una codificación binaria o entera habría que penalizar o reparar dos tipos de restricción; aquí queda una sola, la capacidad, y el valor de λ es fácil de justificar. Además, cualquier vector de genes produce una solución, de modo que el cruce y la mutación nunca generan individuos inválidos.

Las soluciones óptimas del problema de transporte son básicas: usan a lo sumo $n + m - 1$ rutas (12 en el caso base). La mutación de poda empuja al AG hacia ese tipo de solución, porque al llevar un gen a cero cierra una ruta completa. La selección por torneo mantiene una presión selectiva estable aunque los costos de los individuos sean muy parecidos, situación en la que la ruleta pierde poder de discriminación. El elitismo garantiza que el mejor costo nunca empeore entre generaciones.

Estas decisiones se comprobaron con experimentos sobre los datos de prueba (Sección VIII-C, Tabla VIII): con la mutación uniforme el gap promedio sube de 0,52 % a 4,55 %; con la ruleta, a 1,36 %, y sin elitismo, a 4,91 %. Como el AG es estocástico, cada comparación se ejecuta con varias semillas y se reportan el mejor resultado, el promedio y la desviación estándar.

VI. ARQUITECTURA DEL SISTEMA

A. Arquitectura en capas

El software sigue una arquitectura cliente-servidor de tres capas, en la que cada capa se comunica solo con la siguiente:

- **Presentación:** aplicación web en Streamlit [9] con cuatro pantallas: parametrización, resolver y comparar, sensibilidad y reportes. Usa Plotly para las gráficas y httpx para hablar con la API.
- **Lógica:** API REST en FastAPI [10], organizada en controladores, servicios, dominio, motores de optimización y análisis. Recibe y entrega JSON validado con Pydantic.
- **Datos:** base SQLite [11] a la que la lógica accede mediante repositorios y el ORM SQLAlchemy [12]. Guarda la red (agroempresas, plantas, rutas y parámetros) y el historial de comparaciones.

Los modelos de optimización usan solo la biblioteca estándar de Python. Los dos métodos implementan la misma interfaz ISolver (patrón Strategy) y se crean con una fábrica (SolverFactory), de modo que el comparador y el simulador de escenarios los usan sin depender de su implementación. Los servicios reciben sus dependencias por inyección desde un único punto de composición, lo que permite probar cada pieza por separado: el backend tiene 117 pruebas automáticas.

B. Diagrama de aplicaciones

La Fig. 1 muestra las tres capas, sus componentes principales y los protocolos con que se conectan, junto con el contexto del negocio y el flujo de una comparación.

C. Diagrama de clases

La Fig. 3 muestra las clases del dominio y de los motores: la red logística, la instancia de transporte que se construye a partir de ella, las soluciones exacta y genética y la fábrica de solucionadores. El Anexo B reúne los diagramas de los demás paquetes: método exacto, algoritmo genético, análisis, servicios y persistencia, y presentación.

D. Diagramas de secuencia e interfaz

El Anexo A contiene los diagramas de secuencia de los cinco flujos principales. La Fig. 2 muestra la pantalla «Resolver y comparar» tras una comparación: indicadores de costo, gap y tiempo, convergencia del AG, asignación óptima y capacidad usada por planta. La interfaz incluye además un mapa con las rutas del despacho óptimo; con conexión a internet el trazado sigue las carreteras, obtenido del servicio OSRM [13].

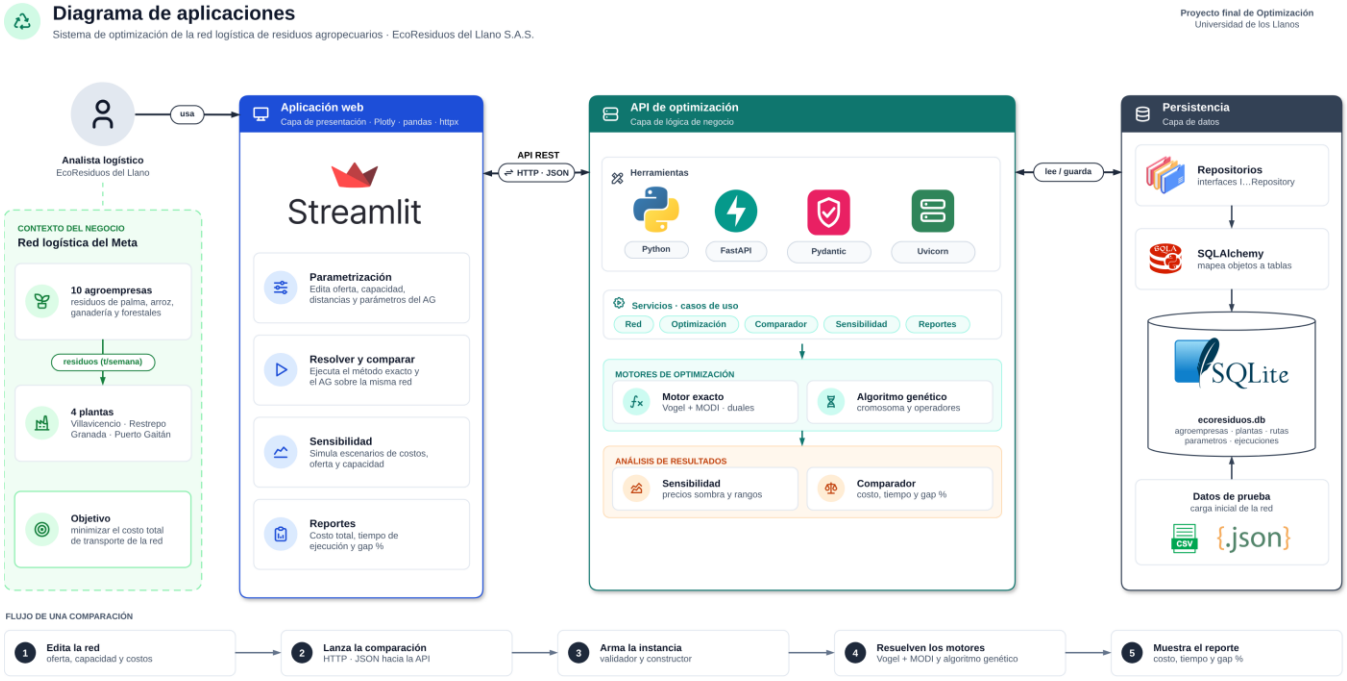


Fig. 1. Diagrama de aplicaciones del sistema: capa de presentación (Streamlit), lógica (FastAPI) y datos (SQLite), con el flujo de una comparación.

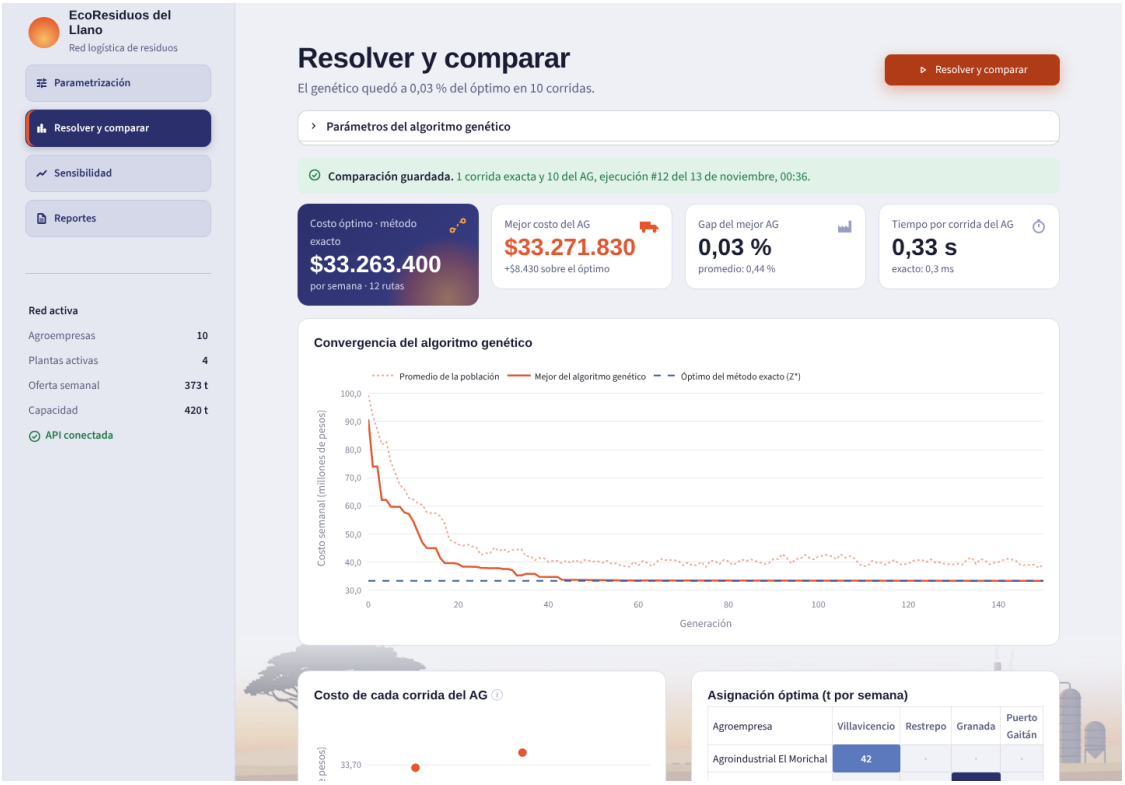


Fig. 2. Pantalla «Resolver y comparar» de la aplicación con los datos de prueba.

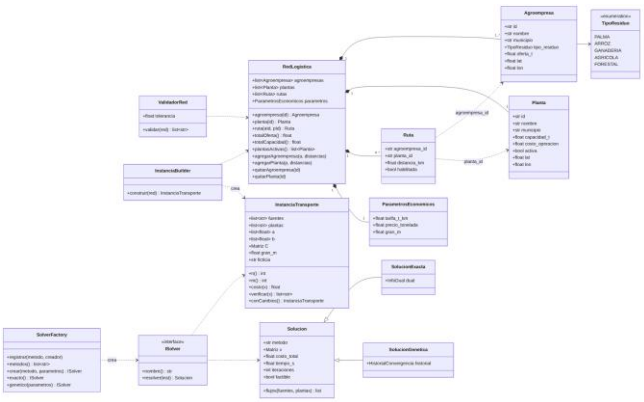


Fig. 3. Diagrama de clases del dominio y de los motores de optimización.

VII. DATOS DE PRUEBA REGIONALES

Los datos de prueba describen una red plausible del departamento del Meta. Las Tablas III y IV resumen las empresas y las plantas. La oferta suma 373 t por semana y la capacidad 420 t, con una holgura total de 47 t (11 %).

TABLA III
EMPRESAS AGROPECUARIAS DE LA RED

ID	Empresa	Municipio	Residuo	Oferta (t)
A01	Agroindustrial El Morichal	Villavicencio	Palma	42
A02	Palmas del Ariari	Granada	Palma	65
A03	Arrocera La Llanura	Puerto López	Arroz	58
A04	Avícola San Martín	San Martín	Ganadería	24
A05	Lácteos Cumaral	Cumaral	Ganadería	18
A06	Porcícola Guamal	Guamal	Ganadería	21
A07	Cítricos Acacías	Acacías	Agrícola	30
A08	Palmar de Castilla	Castilla la Nueva	Palma	47
A09	Arroz Puerto Gaitán	Puerto Gaitán	Arroz	52
A10	Cacao Restrepo	Restrepo	Agrícola	16

TABLA IV
PLANTAS DE PROCESAMIENTO

ID	Municipio	Capacidad (t)	Costo de operación ($/t)
P01	Villavicencio	140	$18.000
P02	Restrepo	70	$15.500
P03	Granada	120	$16.500
P04	Puerto Gaitán	90	$17.200

Las coordenadas de los municipios son aproximadas y la distancia de cada ruta se estimó como la distancia en línea recta multiplicada por 1,3 para aproximar el trazado vial, con un mínimo de 5 km dentro del mismo municipio. La tarifa de transporte es t = \$2.500 por tonelada-kilómetro. La ruta Palmas del Ariari → Puerto Gaitán está deshabilitada para probar el caso de la Gran M. La Tabla V muestra la matriz de costos unitarios que resulta de (1). Estos datos se reemplazarán por los reales del caso cuando estén disponibles.

TABLA V
COSTO UNITARIO DE CADA RUTA (MILES DE $ POR TONELADA)

	Villavicencio	Restrepo	Granada	Puerto Gaitán
A01	30,5	63,5	233,5	577,2
A02	235,0	277,8	29,0	*M*
A03	260,5	242,5	349,5	342,9
A04	181,3	224,5	70,8	641,2

	Villavicencio	Restrepo	Granada	Puerto Gaitán
A05	86,5	43,0	290,0	523,5
A06	125,0	170,5	139,0	643,7
A07	91,5	136,5	177,0	633,2
A08	134,0	177,8	118,0	622,2
A09	578,0	549,0	664,3	29,7
A10	66,0	28,0	278,8	550,7

M = 10^8: ruta deshabilitada.

VIII. RESULTADOS

A. Solución óptima

Vogel entrega una solución inicial de $35.243.400 por semana y MODI la lleva al óptimo, Z^* = $33.263.400, en 2 iteraciones (una mejora de 5,6 %). El cálculo completo tarda 0,43 ms. La solución usa las 12 rutas de la Tabla VI. La tabla balanceada tiene 11 filas y 4 columnas, es decir, 14 celdas básicas; las otras dos son de la fila ficticia y representan la capacidad libre de Villavicencio y Puerto Gaitán.

Restrepo y Granada quedan al tope de su capacidad (70 t y 120 t). Villavicencio usa 131 de 140 t y Puerto Gaitán 52 de 90 t: solo recibe la producción de la arrocera vecina, porque llevar residuos desde el piedemonte cuesta más de $500.000 por tonelada. Solo dos empresas reparten su oferta entre dos plantas: Arrocera La Llanura (Restrepo y Villavicencio) y Palmar de Castilla (Granada y Villavicencio).

TABLA VI
RUTAS DE LA SOLUCIÓN ÓPTIMA

Ruta	t/semana	$/t	Costo semanal
Palmas del Ariari → Granada	65	29.000	$1.885.000
Arroz Puerto Gaitán → Puerto Gaitán	52	29.700	$1.544.400
Agroindustrial El Morichal → Villavicencio	42	30.500	$1.281.000
Arrocera La Llanura → Restrepo	36	242.500	$8.730.000
Palmar de Castilla → Granada	31	118.000	$3.658.000
Cítricos Acacías → Villavicencio	30	91.500	$2.745.000
Avícola San Martín → Granada	24	70.750	$1.698.000
Arrocera La Llanura → Villavicencio	22	260.500	$5.731.000
Porcícola Guamal → Villavicencio	21	125.000	$2.625.000
Lácteos Cumaral → Restrepo	18	43.000	$774.000
Palmar de Castilla → Villavicencio	16	134.000	$2.144.000
Cacao Restrepo → Restrepo	16	28.000	$448.000
Total	373		$33.263.400

B. Comparación con el algoritmo genético

La desviación del AG frente al óptimo se mide con el gap:

$$\text{gap} = (Z_{AG} - Z^*) / Z^* \times 100\ \% \qquad (9)$$

La Tabla VII compara los dos métodos sobre la misma instancia, con la configuración base del AG (población 100, 300 generaciones, torneo, cruce de un punto y mutación de poda) y 30 corridas. En 26 de las 30 corridas el gap es menor a 1 % y la mejor corrida queda $926 por encima del óptimo, con la misma estructura de 12 rutas. El promedio lo empujan pocas corridas que se estancan en un óptimo local; la peor queda a 9,34 %.

TABLA VII
MÉTODO EXACTO FRENTE AL ALGORITMO GENÉTICO (30 CORRIDAS)

Métrica	Exacto	Algoritmo genético
Mejor costo ($/semana)	$33.263.400	$33.264.326
Costo promedio ($/semana)	$33.263.400	$33.435.115
Desviación estándar	—	$569.796
Gap del mejor / promedio	0 %	0,003 % / 0,52 %
Corridas con gap < 1 %	—	26 de 30
Tiempo de cómputo	0,43 ms	0,77 s por corrida
Cumple todas las restricciones	Sí	Sí (mejor corrida)

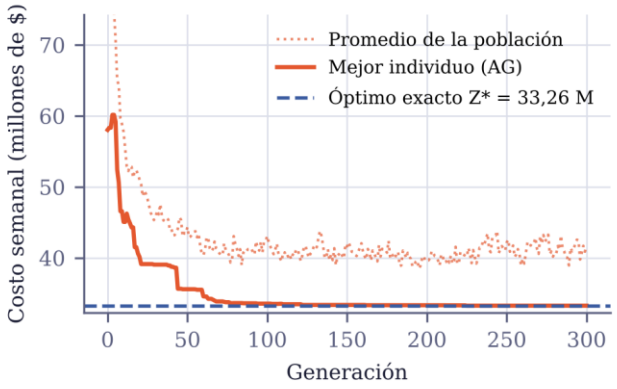


Fig. 4. Convergencia de la mejor corrida del AG (configuración base). La línea discontinua es el óptimo del método exacto.

C. Ajuste de los parámetros del algoritmo genético

Para justificar la configuración se corrieron ocho variantes, cambiando un parámetro a la vez respecto a la base y con las mismas 30 semillas. La Tabla VIII y la Fig. 5 resumen los resultados.

TABLA VIII
DESEMPEÑO DEL AG SEGÚN LA CONFIGURACIÓN (30 CORRIDAS CADA UNA)

Configuración	Mejor gap	Gap promedio	Desviación ($)	Peor gap	Corridas con gap < 1 %	Tiempo por corrida
Base (torneo, 1 punto, poda)	0,003 %	0,52 %	569.796	9,34 %	26 de 30	0,77 s
Selección por ruleta	0,075 %	1,36 %	459.320	5,02 %	17 de 30	0,90 s
Cruce de dos puntos	0,002 %	0,11 %	135.946	2,25 %	29 de 30	0,85 s
Mutación uniforme	2,292 %	4,55 %	489.978	7,10 %	0 de 30	0,82 s
Población 50	0,005 %	1,38 %	868.189	9,35 %	21 de 30	0,42 s
Población 200	0,001 %	0,05 %	36.099	0,53 %	30 de 30	1,61 s
150 generaciones	0,009 %	1,61 %	955.095	10,07 %	22 de 30	0,42 s

Configuración	Mejor gap	Gap promedio	Desviación ($)	Peor gap	Corridas con gap < 1 %	Tiempo por corrida
Sin elitismo	0,322 %	4,91 %	945.480	15,04 %	2 de 30	0,80 s

En «Cruce de dos puntos» y «Población 200», la mejor corrida excede la capacidad de Restrepo en 1 a 3 kg; la mejor solución que cumple todas las restricciones queda en el mismo orden de gap.

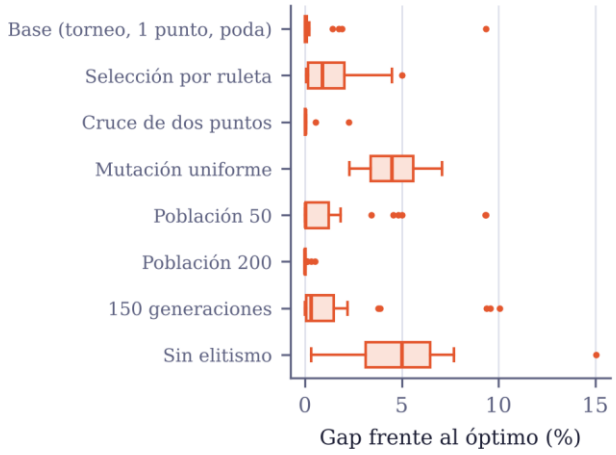


Fig. 5. Distribución del gap de las 30 corridas de cada configuración del AG.

Las tres decisiones de diseño que más pesan son la mutación de poda, el elitismo y el torneo: al cambiar cualquiera de ellas el gap promedio se multiplica entre 2,6 y 9,5 veces. El cruce de dos puntos y una población de 200 mejoran el promedio (0,11 % y 0,05 %), la segunda a costa del doble de tiempo. Con 150 generaciones o población 50 el AG corre en la mitad del tiempo, pero sube la probabilidad de estancarse.

En dos configuraciones la mejor corrida excede la capacidad de Restrepo en 1 a 3 kg. Con la decodificación proporcional es difícil que una planta quede exactamente llena, y una penalización de unos $1.300 por kilogramo (λ = $1.328.500 por tonelada) pesa poco frente a un costo total de $33 millones, así que la selección no termina de eliminar ese exceso en 300 generaciones. Un operador de reparación que recorte el exceso al final de la corrida eliminaría el caso.

IX. ANÁLISIS DE SENSIBILIDAD

El análisis se calcula sobre el tablero óptimo de MODI, sin volver a resolver el modelo, y se complementa con la simulación de escenarios, que sí vuelve a resolver en cada paso (Fig. 13, Anexo A). Los precios sombra y rangos se verificaron volviendo a resolver el modelo con cada cambio en 150 instancias aleatorias.

A. Precios sombra de la capacidad

Con el potencial de la fila ficticia fijado en cero, v_j es el precio sombra de la capacidad de la planta j: indica cuánto cambia el costo total por cada tonelada adicional de capacidad y vale cero en las plantas con capacidad sobrante. Para leerlo como ahorro se reporta $-v_j$. El rango de validez se calcula sobre el camino del árbol de la base que une esa columna con la fila ficticia: las asignaciones del camino cambian en ±Δ y el rango es el intervalo que las mantiene no negativas.

TABLA IX
PRECIOS SOMBRA DE LA CAPACIDAD DE LAS PLANTAS

Planta	Usada / capacidad (t)	Holgura (t)	Ahorro por t adicional	Rango de validez (t)
Villavicencio	131 / 140	9	$0	131 – ∞
Restrepo	70 / 70	0	$18.000	61 – 92
Granada	120 / 120	0	$16.000	111 – 136
Puerto Gaitán	52 / 90	38	$0	52 – ∞

Ampliar Restrepo es la inversión con mayor retorno: cada tonelada adicional ahorra $18.000 por semana hasta llegar a 92 t (22 t más, $396.000 semanales). Granada ahorra $16.000 por tonelada hasta 136 t. Ampliar Villavicencio o Puerto Gaitán no reduce el costo, porque ya tienen capacidad libre.

B. Precio sombra de la oferta

El potencial u_i es el costo marginal de que la empresa i entregue una tonelada más. La Tabla X muestra los valores y el rango de oferta en que se mantienen. Una tonelada adicional de Arrocera La Llanura cuesta $260.500, porque tiene que viajar hasta Villavicencio; una de Arroz Puerto Gaitán solo $29.700, porque la procesa la planta del mismo municipio.

TABLA X
PRECIO SOMBRA DE LA OFERTA DE CADA EMPRESA

Empresa	Oferta (t)	Costo por t adicional	Rango de validez (t)
Agroindustrial El Morichal	42	$30.500	0 – 51
Palmas del Ariari	65	$45.000	49 – 74
Arrocera La Llanura	58	$260.500	36 – 67
Avícola San Martín	24	$86.750	8 – 33
Lácteos Cumaral	18	$61.000	0 – 27
Porcícola Guamal	21	$125.000	0 – 30
Cítricos Acacías	30	$91.500	0 – 39
Palmar de Castilla	47	$134.000	31 – 56
Arroz Puerto Gaitán	52	$29.700	0 – 90
Cacao Restrepo	16	$46.000	0 – 25

C. Rangos de optimalidad de los costos

Para una ruta no básica la solución sigue siendo óptima mientras su costo esté en $[c_{ij} - \bar{d}_{ij}, \infty)$. Para una ruta básica, el rango se obtiene de los ciclos de las celdas no básicas que la contienen: un cambio Δ en su costo modifica el costo reducido de esas celdas en ±Δ, según el signo con que aparece en cada ciclo, y el rango es el intervalo de Δ que mantiene todos los $\bar{d}_{ij} \geq 0$. La Tabla XI muestra los rangos de las rutas usadas.

TABLA XI
RANGO DE OPTIMALIDAD DEL COSTO DE LAS RUTAS USADAS, EN PESOS POR TONELADA

Ruta	Actual	Mínimo	Máximo
Palmas del Ariari → Granada	29.000	—	219.000

Ruta	Actual	Mínimo	Máximo
Arroz Puerto Gaitán → Puerto Gaitán	29.700	—	567.000
Agroindustrial El Morichal → Villavicencio	30.500	—	81.500
Arrocera La Llanura → Restrepo	242.500	222.500	260.500
Palmar de Castilla → Granada	118.000	23.500	134.000
Cítricos Acacías → Villavicencio	91.500	—	154.500
Avícola San Martín → Granada	70.750	—	165.250
Arrocera La Llanura → Villavicencio	260.500	242.500	280.500
Porcícola Guamal → Villavicencio	125.000	—	155.000
Lácteos Cumaral → Restrepo	43.000	—	68.500
Palmar de Castilla → Villavicencio	134.000	118.000	195.750
Cacao Restrepo → Restrepo	28.000	—	48.000

— : sin límite inferior; la ruta seguiría siendo parte del óptimo aunque su costo bajara a cero.

Las rutas más estables son las de cada empresa a la planta de su propio municipio: Arroz Puerto Gaitán podría costar hasta $567.000 por tonelada sin que cambie el plan. Las dos rutas de Arrocera La Llanura son las más sensibles, con márgenes de unos $20.000 por tonelada: un cambio de tarifa en la vía Puerto López – Villavicencio alteraría el reparto entre Restrepo y Villavicencio.

D. Simulación de escenarios

Para cambios mayores, el simulador aplica una variación porcentual a un parámetro, vuelve a resolver el modelo en cada paso y registra el costo total y si cambió el conjunto de rutas utilizadas. La Tabla XII resume los escenarios evaluados y las Fig. 6 y 7 muestran las curvas completas.

TABLA XII
ESCENARIOS EVALUADOS

Escenario	Costo ($/semana)	Variación	¿Cambian las rutas?
Base	$33.263.400	0 %	—
Tarifa de transporte +15 %	$38.252.910	+15,0 %	No
Tarifa de transporte −20 %	$26.610.720	-20,0 %	No
Oferta de todas +10 %	$38.812.335	+16,7 %	Sí
Oferta de todas +15 %	Sin solución	—	—
Oferta de todas −20 %	$26.153.920	-21,4 %	Sí
Oferta de Agroindustrial El Morichal +20 %	$33.519.600	+0,8 %	No
Oferta de Palmas del Ariari +20 %	$34.178.200	+2,8 %	Sí
Oferta de Palmar de Castilla +20 %	$34.555.980	+3,9 %	Sí
Capacidad de Restrepo +30 %	$32.885.400	-1,1 %	No
Capacidad de Granada −30 %	$36.793.050	+10,6 %	Sí
Capacidad de Villavicencio −30 %	$36.545.250	+9,9 %	Sí
Cierre de la planta de Villavicencio	Sin solución	—	—
Cierre de la planta de Restrepo	Sin solución	—	—
Cierre de la planta de Granada	Sin solución	—	—
Cierre de la planta de Puerto Gaitán	Sin solución	—	—

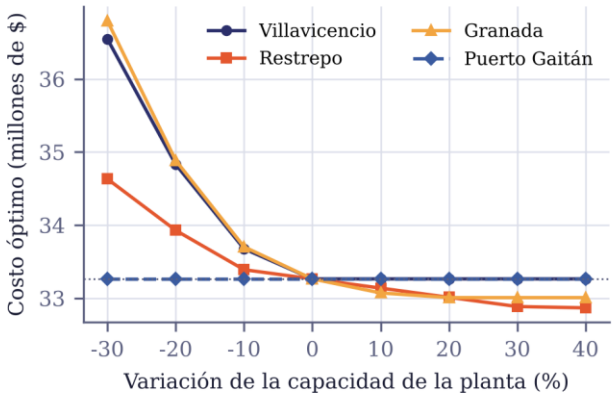


Fig. 6. Costo óptimo al variar la capacidad de cada planta entre −30 % y +40 %.

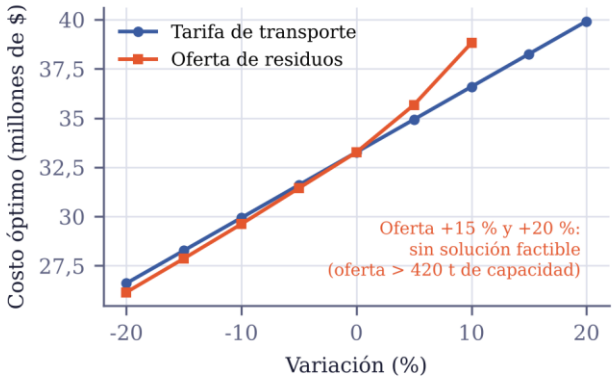


Fig. 7. Costo óptimo al variar la tarifa de transporte y la oferta de todas las empresas.

Un alza de 15 % en la tarifa de transporte, por ejemplo por el precio del combustible, eleva el costo a $38.252.910 (+15,0 %) sin cambiar ninguna ruta: el plan óptimo es robusto ante el costo del transporte entre −20 % y +20 %. La oferta es el parámetro más crítico. Con 10 % más de residuos el costo sube 16,7 %, más que proporcionalmente, porque las plantas cercanas se llenan y el excedente debe viajar más lejos; desde +15 % la oferta (429 t) supera la capacidad total y el problema deja de tener solución.

La red tampoco resiste el cierre de ninguna planta: sin Puerto Gaitán, la de menor uso, quedan 330 t de capacidad para 373 t de oferta. Reducir 30 % la capacidad de Granada o de Villavicencio encarece el despacho entre 9,8 % y 10,6 %, mientras que Puerto Gaitán puede perder hasta 30 % de capacidad sin efecto en el costo.

X. ANÁLISIS COMPARATIVO Y DISCUSIÓN

A. Calidad de la solución

Con los datos de prueba regionales el AG encuentra soluciones a menos de 0,01 % del óptimo, pero no en todas las corridas: en la configuración base 4 de 30 terminan por encima de 1 %. El método exacto entrega siempre el óptimo y además los valores duales que hacen posible el análisis de sensibilidad, algo que el AG no ofrece.

B. Tiempo de cómputo y escalabilidad

Para medir la escalabilidad se generaron instancias aleatorias de mayor tamaño, con la misma proporción de holgura que el caso base (12 %), y se resolvieron con ambos métodos usando la configuración base del AG (Tabla XIII y Fig. 8).

TABLA XIII
ESCALABILIDAD DE LOS MÉTODOS (PROMEDIO DE 3 INSTANCIAS)

n × m	Variables	Exacto (s)	Iter. MODI	AG (s)	Gap AG	AG factible
10 × 4	40	0,0008	3	0,9	0,6 %	2 de 3
20 × 6	120	0,0014	8	2,0	18,0 %	2 de 3
40 × 8	320	0,0054	15	4,9	82,5 %	2 de 3
80 × 12	960	0,0216	24	13,9	219,3 %	1 de 3

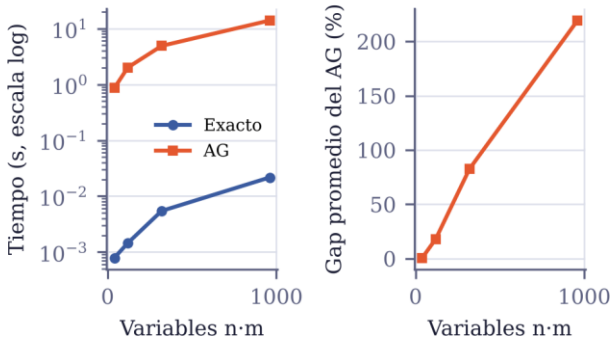


Fig. 8. Tiempo de cómputo (escala logarítmica) y gap del AG según el tamaño de la instancia.

El método exacto escala muy bien: con 960 variables tarda 22 ms. El AG, con población y generaciones fijas, tarda cientos de veces más y su calidad cae rápido: con 80 empresas y 12 plantas el gap promedio supera el 200 % y solo una de las tres corridas respeta la capacidad; en los demás tamaños falla una de cada tres. Para instancias grandes el AG necesitaría más generaciones, una población proporcional al número de genes y un operador de reparación.

C. Respuesta ante escenarios

El simulador usa el método exacto, que resuelve cada paso en menos de un milisegundo e indica si cambia la base. Con el AG cada paso costaría varias corridas y el cambio de ruta podría confundirse con la variación aleatoria entre semillas.

TABLA XIV
COMPARACIÓN CUALITATIVA DE LOS MÉTODOS

Criterio	Método exacto	Algoritmo genético
Calidad	Óptimo garantizado	Cercana al óptimo en instancias pequeñas; variable
Tiempo	Milisegundos	Segundos por corrida; crece con el tamaño
Información dual	Precios sombra y rangos	No disponible
Restricciones	Siempre las cumple	Por penalización; puede excederlas levemente
Flexibilidad	Solo modelos lineales de transporte	Admite costos no lineales, rutas con costo fijo u objetivos múltiples

El AG tiene sentido como base para extensiones que rompen la linealidad del modelo, como costos fijos por abrir una ruta, descuentos por volumen o restricciones de flota, donde Vogel y MODI ya no aplican. Para el problema tal como está planteado, el método exacto es la opción adecuada para la operación diaria.

XI. CONCLUSIONES

- El modelo de transporte resuelto con Vogel y MODI encuentra el plan de despacho de costo mínimo, $33.263.400 por semana con 12 rutas, en menos de un milisegundo y con la información dual necesaria para la sensibilidad.
- El AG diseñado (codificación real, decodificación proporcional, torneo, mutación de poda y elitismo) llega a 0,003 % del óptimo en su mejor corrida y a 0,52 % en promedio. Los experimentos confirman que cada una de esas decisiones mejora el resultado.
- Restrepo y Granada limitan la red. Ampliar Restrepo en 22 t ahorraría $396.000 por semana, y es la primera inversión que conviene evaluar.
- La red no tiene margen ante el cierre de una planta ni ante un aumento de la oferta mayor al 12 %. Antes de vincular nuevas empresas hay que ampliar la capacidad.
- La arquitectura en tres capas, con una interfaz común para los solucionadores, permitió comparar ambos métodos y simular escenarios sin acoplar la interfaz al algoritmo.

Como trabajo futuro se propone reemplazar los datos de prueba por los reales del caso, agregar un operador de reparación al AG y extender el modelo con costos fijos por ruta, donde la metaheurística aporta más que el método exacto.

REFERENCIAS

[1] Universidad de los Llanos, “Proyecto final de curso: optimización de la red logística de recepción y procesamiento de residuos agropecuarios utilizando modelos de redes y algoritmos genéticos,” asignatura Optimización, Programa de Ingeniería de Sistemas, Villavicencio, Colombia, 2026.

[2] H. A. Taha, *Operations Research: An Introduction*, 10th ed. Pearson, 2017.

[3] F. S. Hillier y G. J. Lieberman, *Introduction to Operations Research*, 10th ed. New York, NY, USA: McGraw-Hill, 2015.

[4] N. V. Reinfeld y W. R. Vogel, *Mathematical Programming*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1958.

[5] G. B. Dantzig, “Application of the simplex method to a transportation problem,” en *Activity Analysis of Production and Allocation*, T. C. Koopmans, Ed. New York, NY, USA: Wiley, 1951, pp. 359–373.

[6] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: Univ. of Michigan Press, 1975.

[7] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.

[8] Z. Michalewicz, *Genetic Algorithms + Data Structures = Evolution Programs*, 3rd ed. Berlin, Germany: Springer, 1996.

[9] Streamlit, “Streamlit documentation.” [En línea]. Disponible: https://docs.streamlit.io

[10] S. Ramírez, “FastAPI.” [En línea]. Disponible: https://fastapi.tiangolo.com

[11] SQLite, “SQLite documentation.” [En línea]. Disponible: https://www.sqlite.org/docs.html

[12] SQLAlchemy, “SQLAlchemy documentation.” [En línea]. Disponible: https://docs.sqlalchemy.org

[13] D. Luxen y C. Vetter, “Real-time routing with OpenStreetMap data,” en *Proc. 19th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, Chicago, IL, USA, 2011, pp. 513–516.

ANEXO A. Diagramas de secuencia

El analista edita los datos de la red. La interfaz envía los cambios a la API; el servicio los aplica, ValidadorRed comprueba las reglas del negocio y solo una red válida se guarda en la base de datos.

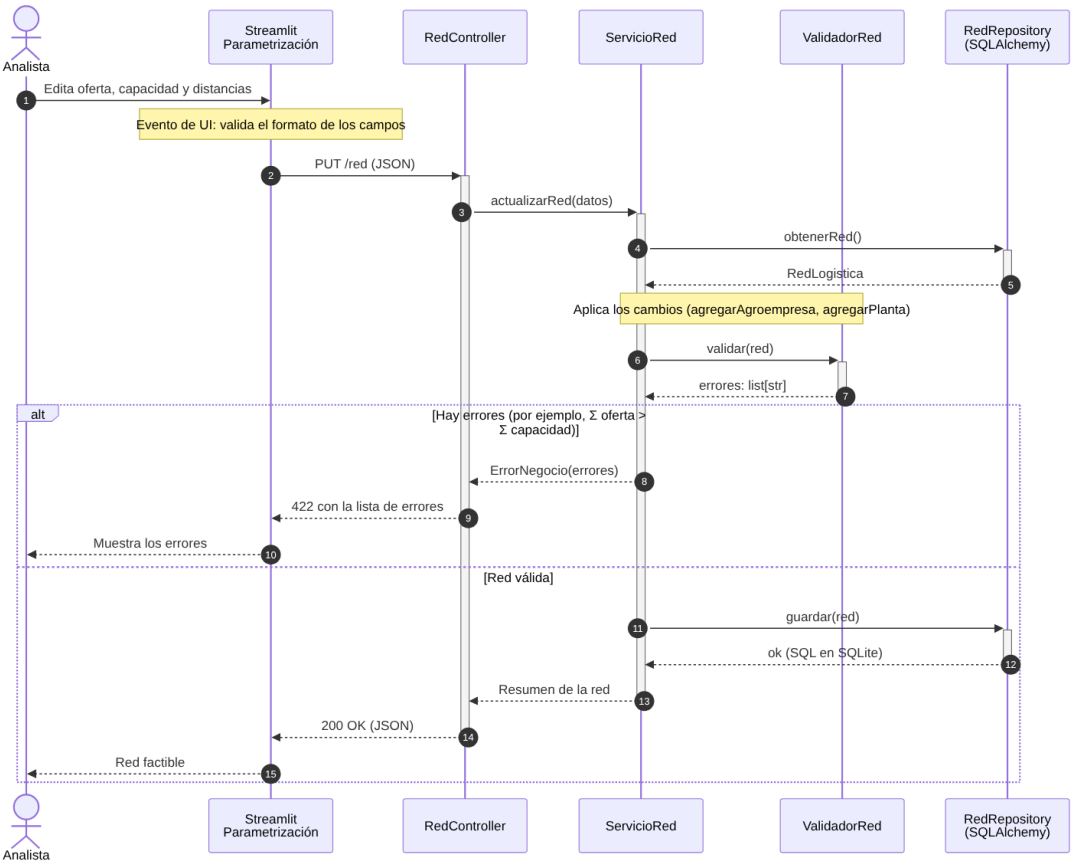


Fig. 9. Diagrama de secuencia de la parametrización y validación de la red.

El servicio arma la instancia y el comparador ejecuta el método exacto una vez y el AG con varias semillas. Con esos resultados calcula el mejor costo, el promedio, la desviación y el gap, y la comparación se guarda en el historial.

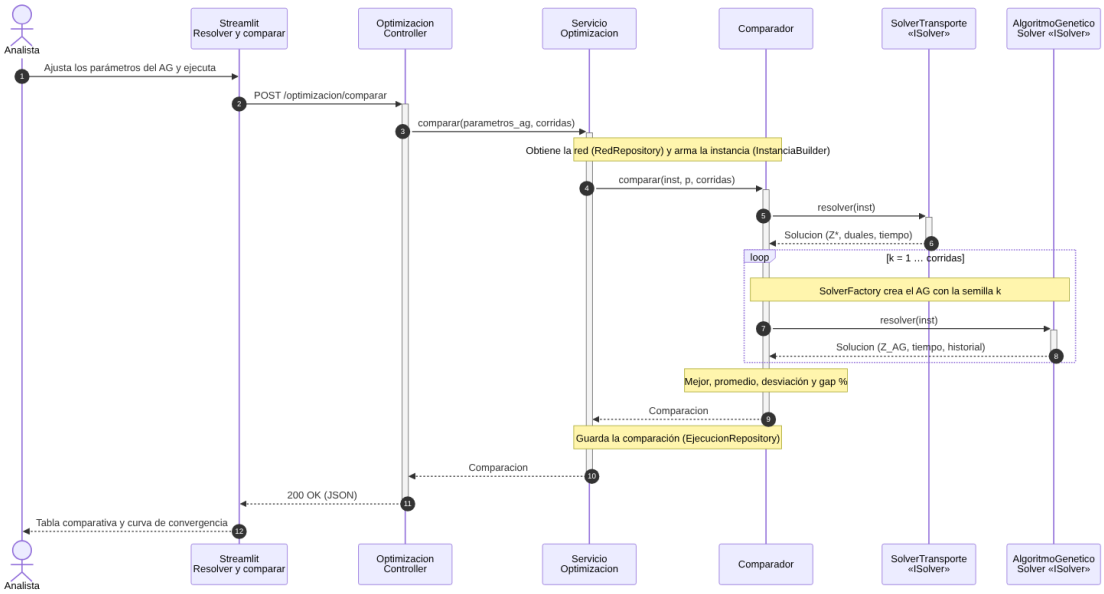


Fig. 10. Diagrama de secuencia de la comparación entre el método exacto y el algoritmo genético.

El método exacto balancea la instancia, obtiene la solución inicial con Vogel y la mejora con MODI hasta que todos los costos reducidos son no negativos. Los valores duales del tablero óptimo se entregan junto con la solución.

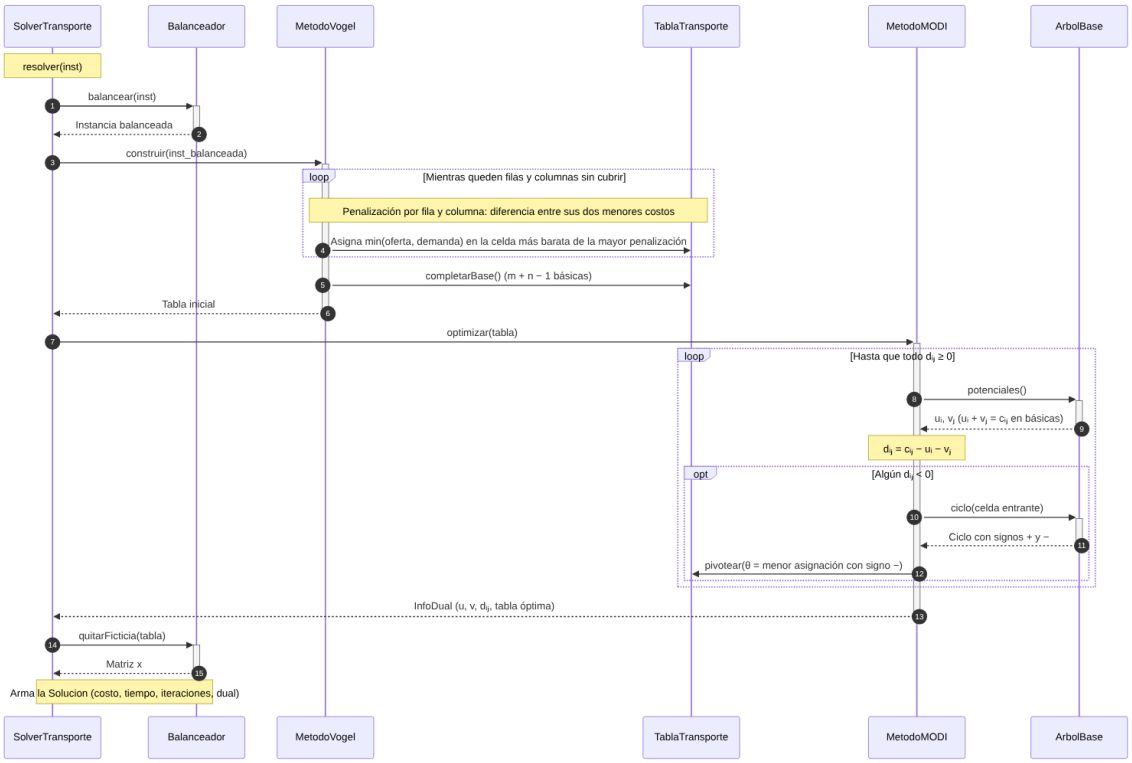


Fig. 11. Diagrama de secuencia del método exacto (Vogel y MODI).

El algoritmo evalúa la población inicial y repite en cada generación la selección, el cruce, la mutación y la evaluación de los hijos. Al final decodifica el mejor individuo y verifica las restricciones.

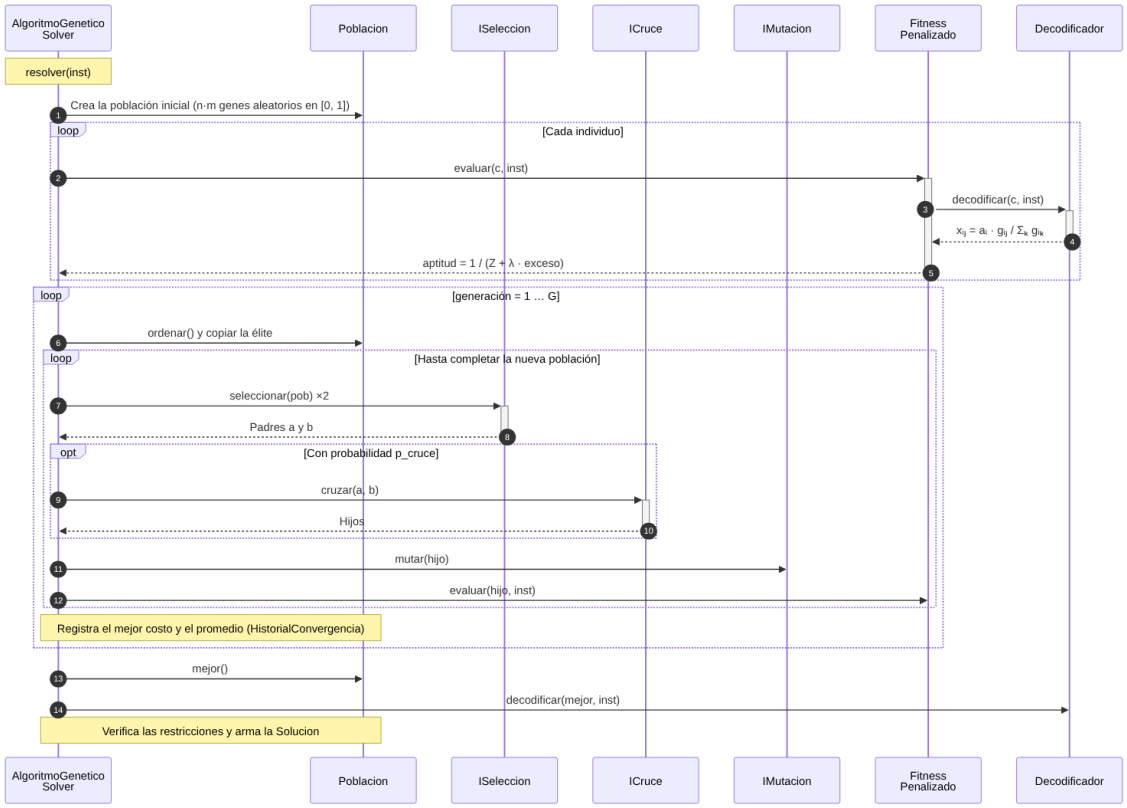


Fig. 12. Diagrama de secuencia del algoritmo genético.

Con la solución óptima, el analizador calcula los precios sombra y los rangos sin volver a resolver. El simulador aplica cada paso del escenario, resuelve la instancia modificada y registra el costo y si cambió la base.

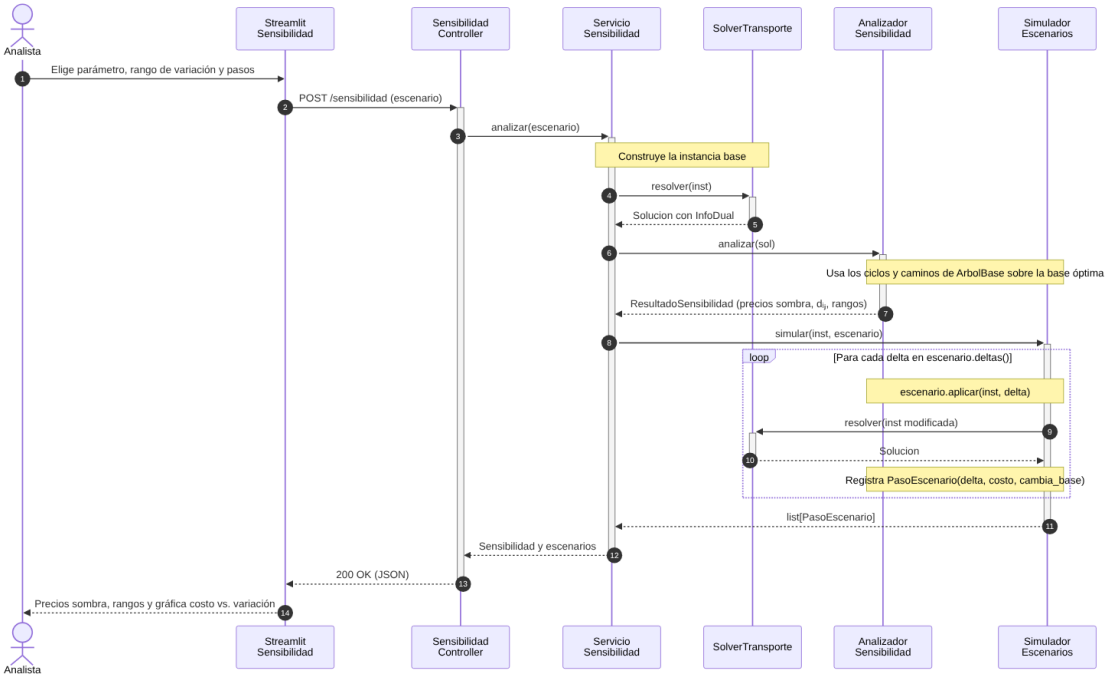


Fig. 13. Diagrama de secuencia del análisis de sensibilidad y la simulación de escenarios.

ANEXO B. Diagramas de clases

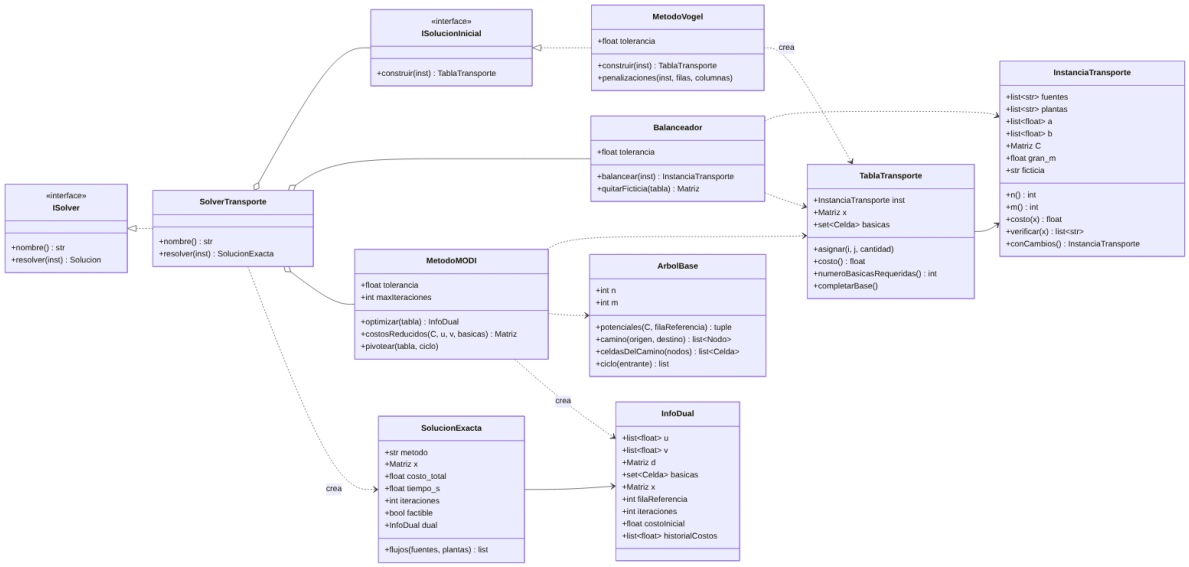


Fig. 14. Clases del método exacto (Vogel y MODI).

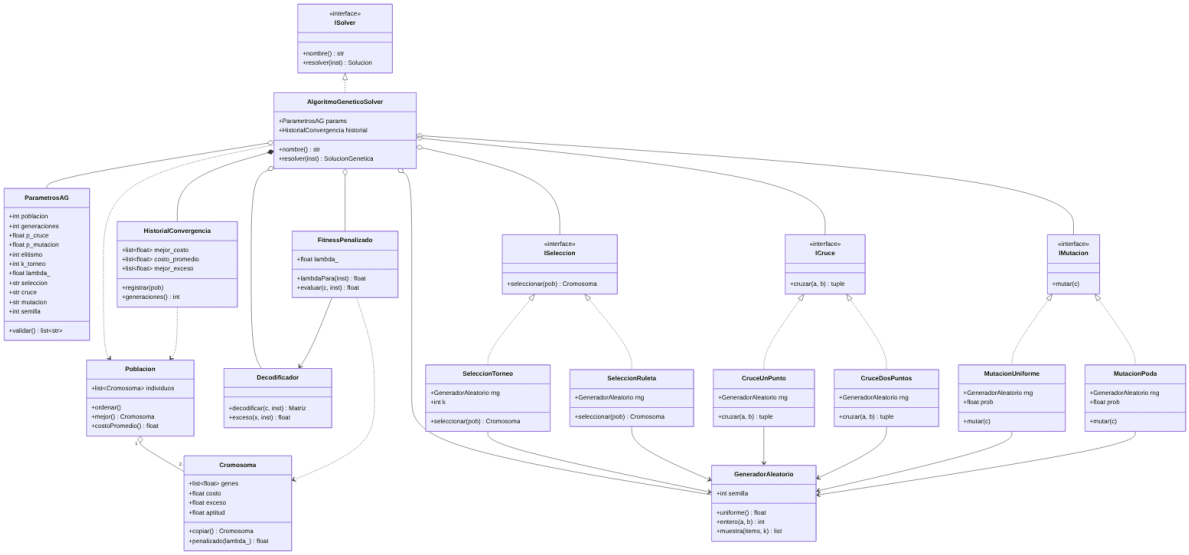


Fig. 15. Clases del algoritmo genético.

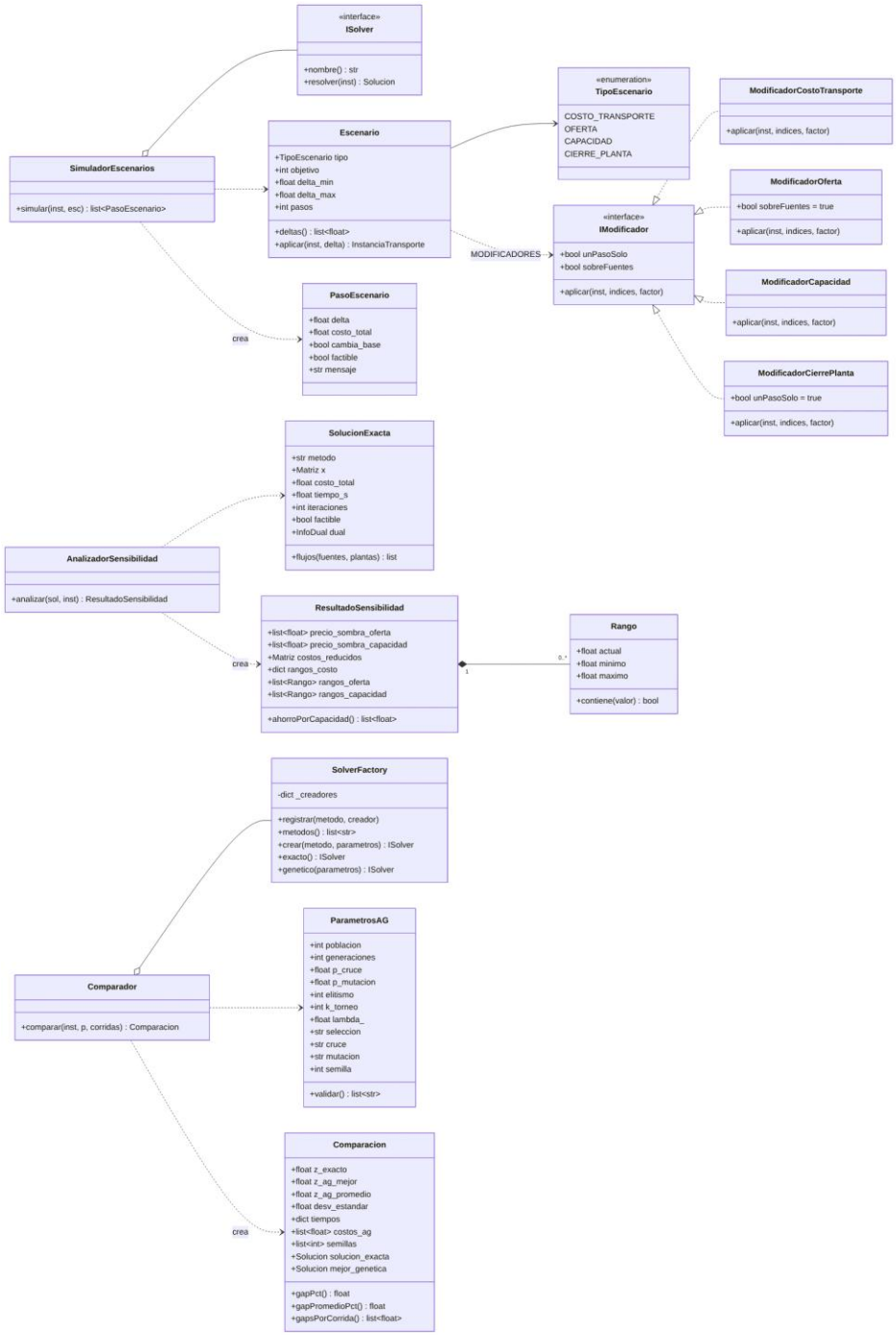


Fig. 16. Clases del análisis de sensibilidad, escenarios y comparación.

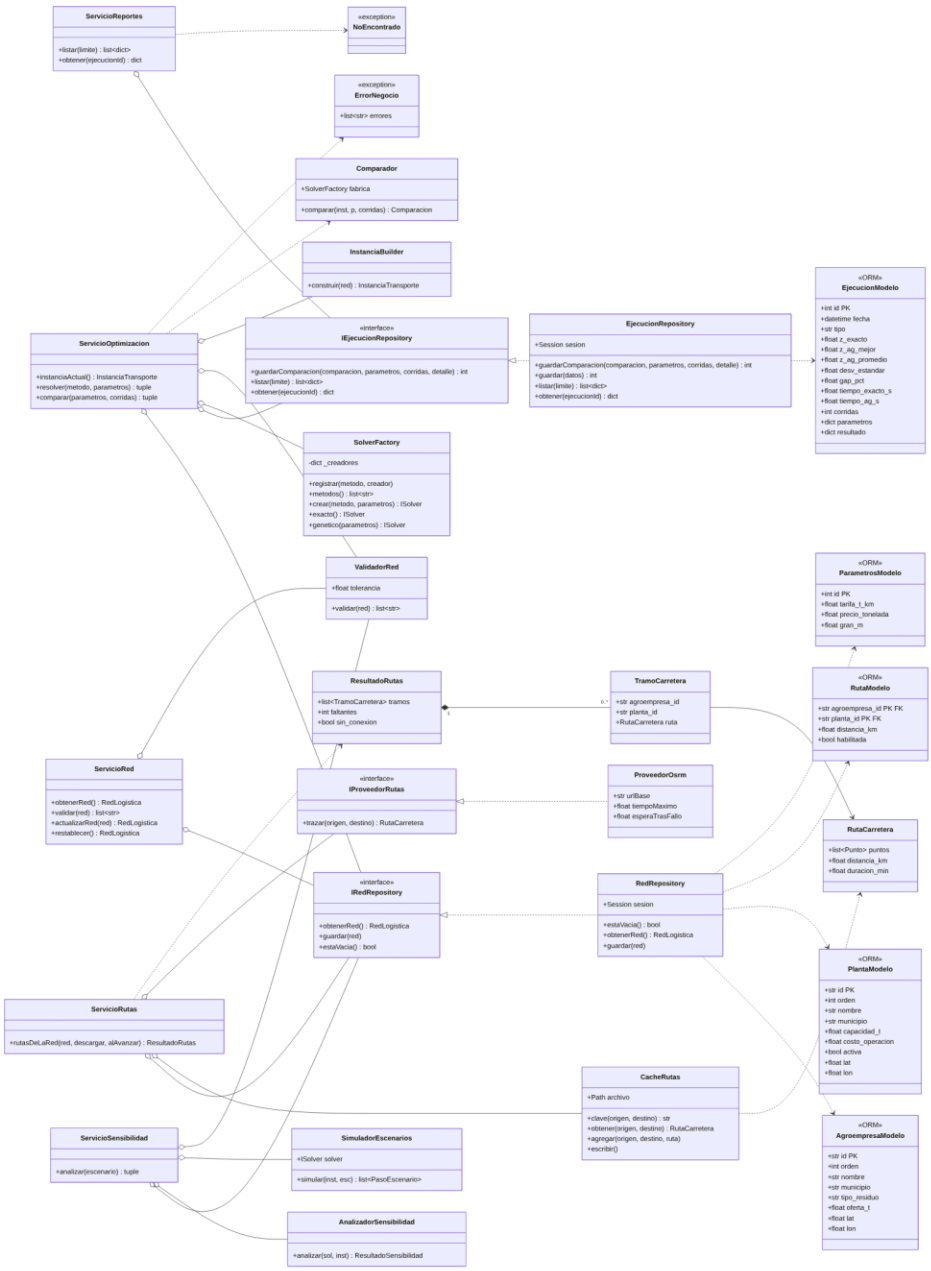


Fig. 17. Clases de servicios y persistencia.

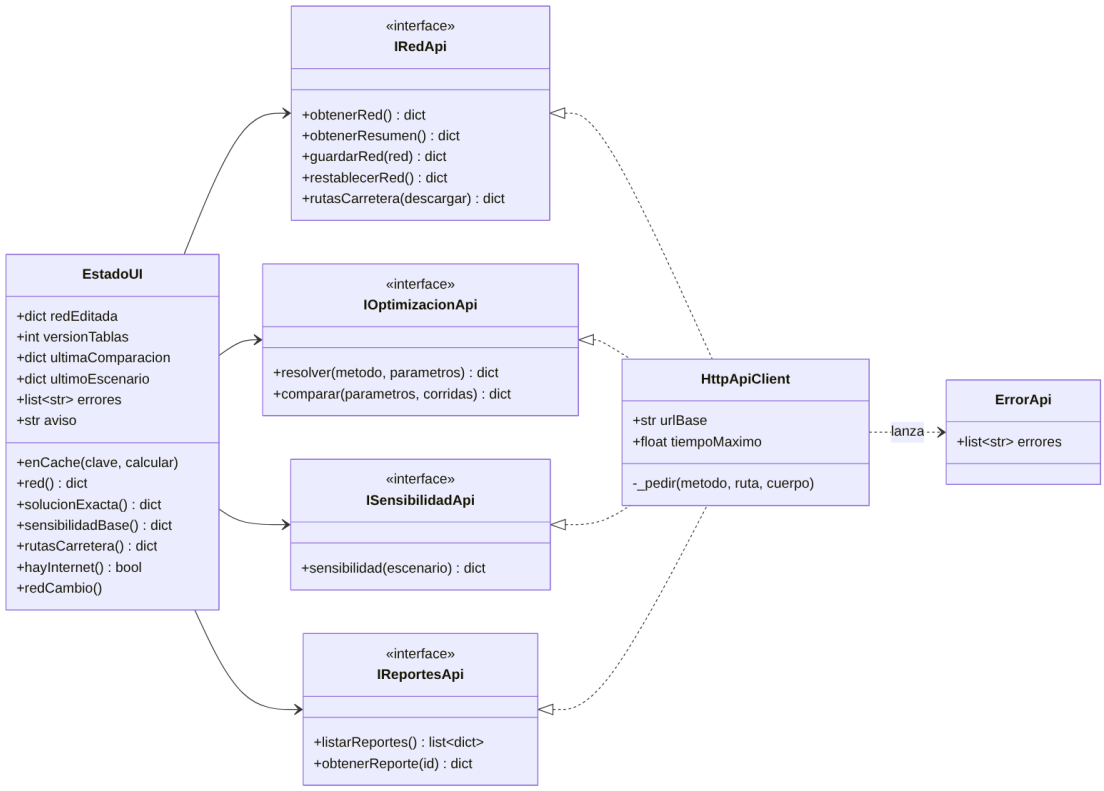


Fig. 18. Clases de la capa de presentación.

ANEXO C. Código fuente y ejecución

El código fuente completo está en el repositorio https://github.com/Eudamone/Proy-Optimizacion-Evolutics. Para ejecutarlo se instalan las dependencias con «pip install -r requirements.txt»; luego se inicia la API con «uvicorn app.main:app» desde la carpeta backend y la interfaz con «streamlit run app.py» desde la carpeta frontend. Las pruebas se ejecutan con «pytest» desde la carpeta backend.